UNIDAD 7

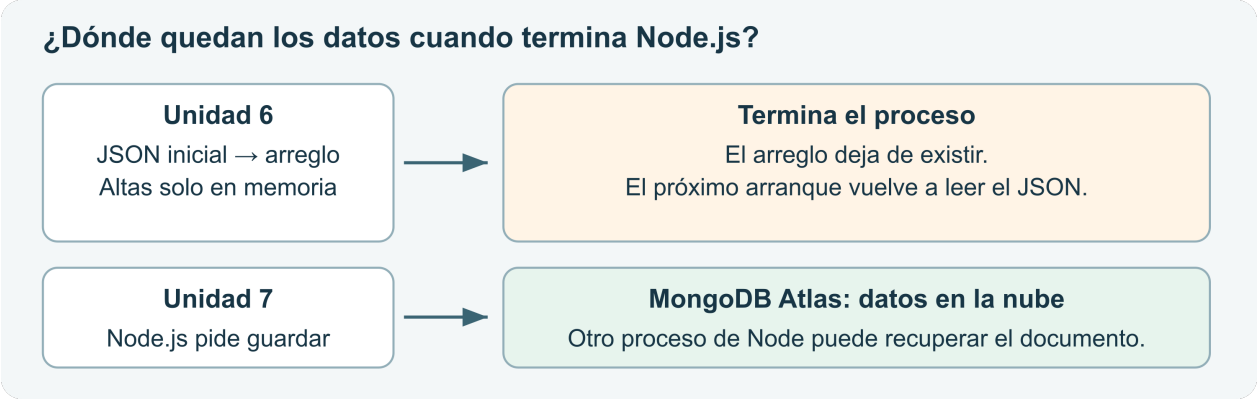
Clase 01 - Primeros documentos persistentes con MongoDB

1. De dónde venimos y qué problema vamos a resolver

En la unidad 5 trabajábamos principalmente dentro de `src/index.js` . En la unidad 6 separamos arranque, aplicación, rutas, controladores, servicio y middleware. No descartamos ese trabajo: ahora copiamos **ese proyecto modular terminado**, con la corrección de la validación que admite un cuerpo ausente.

El servicio actual recibe productos desde un JSON y trabaja con un arreglo. Un POST agrega un elemento a ese arreglo, pero no modifica el JSON. Al cerrar Node.js, desaparece el arreglo de ese proceso; el siguiente arranque vuelve a leer la semilla. Haber dividido el código mejora la organización, pero no crea persistencia.

MongoDB introduce otro proceso que administra los datos. En las clases ese servidor estará en MongoDB Atlas, en la nube. Node.js le solicita operaciones a través de Internet y espera su resultado. Una escritura confirmada no depende de que siga vivo el proceso de nuestra aplicación.



El arreglo pertenece al proceso de Node; los documentos se conservan en MongoDB aunque termine ese proceso.

Pregunta inicial: si el producto aparece en pantalla y cerramos la aplicación, ¿alcanza eso para afirmar que quedó guardado? No: debemos recuperarlo desde una nueva ejecución, sin volver a crearlo.

2. Vocabulario mínimo para entender lo que hacemos

Concepto	Significado en este ejemplo
Base de datos	`mercado_artesanal_u7` : espacio que agrupa nuestras colecciones
Colección	`productos` : conjunto de documentos del catálogo
Documento	Un producto con campos y valores
Campo	`nombre` , `categoria` , `precio` o `descripcion`
`_id`	Identificador único del documento dentro de su colección
BSON	Formato binario de los documentos; tiene tipos como ObjectId y Date

Un objeto JavaScript vive en nuestra aplicación. Un documento vive en la base de datos. Se parecen en su representación, pero no son el mismo objeto compartido.

Relacional y documental: comparación inicial

En un modelo relacional solemos organizar datos en tablas, filas y columnas, y expresar relaciones mediante claves. En un modelo documental organizamos documentos en colecciones y podemos representar estructuras anidadas. Son modelos distintos, no una clasificación de «viejo» y «nuevo» o «lento» y «rápido».

Un catálogo con productos que tienen atributos variables permite discutir el enfoque documental. Una aplicación con reglas relacionales también puede resolverse con distintas tecnologías: la elección exige analizar consultas, consistencia y mantenimiento. En este curso usamos MongoDB para aprender un modelo documental; no afirmamos que sea la mejor opción para cualquier proyecto.

Que MongoDB admita documentos de distintas formas no significa que nuestra aplicación deba aceptar cualquier dato. Vamos a declarar un contrato en Mongoose. Ese contrato actúa en nuestra aplicación; **no configura por sí mismo reglas de validación en el servidor MongoDB para otros clientes**.

No todo lo que parece JSON es JSON

Este es un **ejemplo ilustrativo**, no un archivo que haya que crear:

```
{
  "_id": "507f1f77bcf86cd799439011",
  "nombre": "Mate de cerámica",
  "categoria": "Cerámica",
  "precio": 12000,
  "descripcion": "Pieza artesanal esmaltada."
}
```

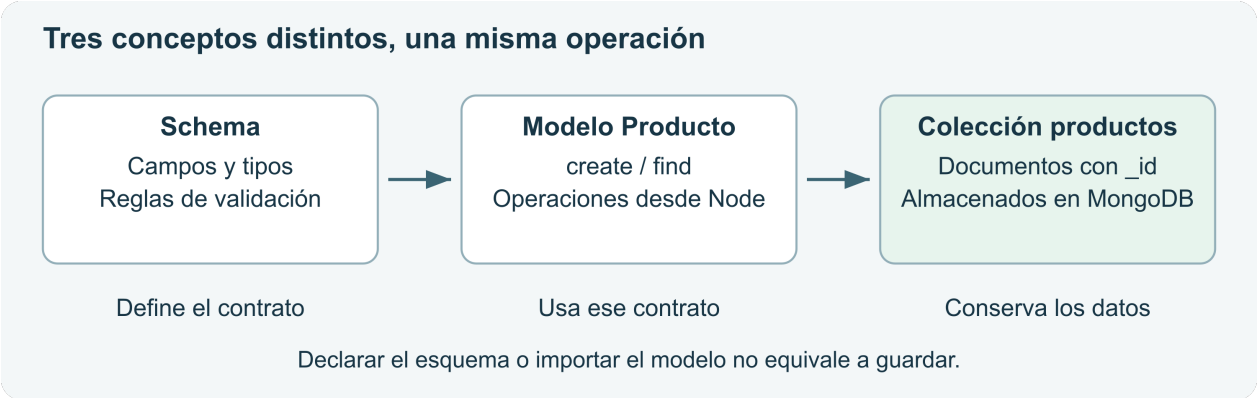
Aquí representamos el identificador como texto para leerlo. En MongoDB, nuestro `_id` será un ObjectId, un tipo BSON que habitualmente se muestra como 24 caracteres hexadecimales. No es el número consecutivo que calculábamos con `reduce` . No debemos aplicarle `Number()` ni incrementarlo.

Mongoose puede asignar `_id` al construir un documento **antes de guardarlo**. Tener un identificador no demuestra que exista en la base. Podemos ver además `__v` , la clave de versión que Mongoose incorpora; no es el ID de negocio.

Referencias: bases y colecciones y tipos BSON.

3. MongoDB, Atlas y Mongoose: qué hace cada herramienta

- **MongoDB Server:** proceso que recibe operaciones y administra el almacenamiento.
- **MongoDB Atlas:** servicio que administra MongoDB en la nube. Usaremos su clúster gratuito Free (M0), pensado para aprendizaje y proyectos pequeños.
- **Data Explorer de Atlas:** interfaz web para inspeccionar los documentos del clúster desde el navegador.
- **Compass:** cliente gráfico opcional para inspeccionar una base. Instalar solo Compass no instala ni inicia un servidor.
- **Mongoose:** paquete de Node.js que permite definir esquemas, construir modelos y operar documentos. Es un ODM: relaciona objetos de la aplicación con documentos.
- **Nuestra aplicación:** usa Mongoose para comunicarse con el servidor.



El esquema define las reglas; el modelo ofrece operaciones; MongoDB almacena los documentos en una colección.

Un `Schema` describe campos y reglas. Un `Model` ofrece operaciones como `create` y `find` . Un documento es una instancia concreta, como el mate que guardaremos. Importar el modelo no equivale a conectarse ni a insertar.

Referencia: modelos de Mongoose.

4. Acuerdo de trabajo: un laboratorio dentro de la copia

Esta unidad incorpora `scripts/` en la raíz para programas cortos ejecutados desde la terminal. Reservamos `src/` para módulos de la aplicación y agrupamos la configuración en `src/config/` . Así distinguimos entradas de consola de la entrada HTTP, sin mezclar scripts con rutas o controladores. Es una convención de organización, no una estructura obligatoria de Express.

No reemplazamos todavía el servicio HTTP en memoria. `src/app.js` , rutas, controladores, vistas, middleware, JSON y recursos públicos conservan su código de la unidad 6. En el paso 2 solo actualizamos la importación de configuración en `src/index.js` . El servidor web sigue arrancando con `npm start` . Los documentos creados por el laboratorio **no aparecerán todavía en la página /productos**.

En la unidad 8 conectaremos las rutas con operaciones asíncronas de MongoDB. Cambiar hoy el servicio sin adaptar sus consumidores haría que un controlador esperase un arreglo pero recibiese una promesa, y que tratase un ObjectId como ID numérico. No es una sustitución automática.

Al final de cada paso aparece el contenido completo de cada archivo de código nuevo o modificado. Los archivos no mencionados se conservan. El lockfile lo genera NPM: no se copia ni edita a mano.

5. Paso 1 - Copiar el proyecto y preparar el entorno

Qué hacemos y por qué

1. Copiar `mercado-artesanal-modular` como `mercado-artesanal-mongodb` . Conservar `src` , `views` , `public` , `datos` , `package.json` , `package-lock.json` , `eslint.config.js` , `.gitignore` y `.env.example` .
2. No copiar `node_modules` , `.git` ni un `.env` con credenciales. Abrir la copia en VS Code.
3. Comprobar `node --version` . Para este material usar Node.js 22.13 o superior compatible con las herramientas de la unidad 6; el entorno de comprobación utiliza Node 24. Mongoose 9 requiere Node 20.19 o superior, pero ese mínimo por sí solo no garantiza compatibilidad con todas las herramientas del proyecto.
4. Ejecutar `npm install` y `npm start` . Revisar inicio, listado, formulario y 404. Detener solo ese servidor con Ctrl+C.
5. Instalar en **la copia**:

```
npm install mongoose@9.10.3 --save-exact
```

Esto modifica `package.json` y `package-lock.json` , y agrega dependencias en `node_modules` . **No instala MongoDB Server.** No necesitamos instalar `dotenv` : ya usamos la opción de Node `--env-file` .

Preparar MongoDB Atlas gratuito para las clases

El recorrido de la unidad utiliza **Atlas Free**, también identificado como **M0**. Cada estudiante trabaja con su propio clúster de aprendizaje. Un clúster es la instancia administrada que aloja nuestras bases: no necesitamos crear uno diferente para cada ejercicio.

1. Acceder a Atlas y preparar el proyecto

Entrar a MongoDB Atlas y crear o utilizar una cuenta. Completar la verificación que solicite el servicio. Dentro de Atlas, crear o seleccionar un proyecto de aprendizaje, por ejemplo `curso-nodejs` . Un proyecto de Atlas agrupa recursos de la nube; no es la carpeta del código ni el repositorio de GitHub.

2. Crear el clúster gratuito

Desde la vista del proyecto, elegir la creación de un clúster y seleccionar **Free / M0**. Elegir uno de los proveedores y regiones disponibles para esa opción, asignar un nombre, por ejemplo `curso-nodejs` , y confirmar la creación. Si ya existe un clúster Free en ese proyecto, reutilizarlo: se admite uno por proyecto.

Verificar que se seleccionó Free antes de confirmar. Flex y los clústeres Dedicated son opciones diferentes y no forman parte de esta práctica. No hace falta cargar datos de ejemplo: guardaremos nuestros propios productos. Esperar a que Atlas indique que el clúster está disponible. Los nombres y posiciones de los botones pueden cambiar; buscar la opción gratuita y la función indicada. Referencia: crear un clúster Free.

3. Crear el usuario de base de datos

En la configuración de acceso a la base, crear un usuario con contraseña. Este usuario lo utiliza Node.js para operar documentos: **es diferente de la cuenta con la que entramos al panel de Atlas**. Guardar su contraseña de manera privada.

Asignar permisos de lectura y escritura sobre las bases del laboratorio. En un clúster personal usado exclusivamente para el curso se puede utilizar la opción de lectura y escritura en todas sus bases; no se necesitan permisos de administrador de Atlas. Si la institución proporciona un proyecto, utilizar los permisos asignados por el docente. Referencia: usuarios de base de datos.

4. Permitir la IP de conexión

En **Network Access / IP Access List**, agregar la IP pública actual de la conexión con la que trabajamos; el asistente puede ofrecer **Add My Current IP Address**. Esperar a que la regla esté activa. Si cambiamos de red, puede ser necesario actualizarla. Mantener el acceso limitado a esa IP, sin elegir acceso desde cualquier dirección. Referencia: requisitos de conexión a Atlas.

5. Obtener la URI para Node.js

En el clúster, abrir **Connect -> Drivers** y seleccionar Node.js. Copiar la cadena de conexión ofrecida por Atlas. Mongoose usa el driver de MongoDB internamente; alcanza con la dependencia Mongoose que instalamos.

En esa cadena reemplazar los marcadores del usuario y la contraseña, y colocar `mercado_artesanal_u7` como nombre de base después del host y antes de `?` . Conservar las opciones de conexión que devuelve Atlas. El host se copia del clúster real: no se escribe `127.0.0.1` para este recorrido.

Si la contraseña contiene caracteres reservados de una URI, codificarlos en ese componente: por ejemplo, `@` se representa como `%40` . No codificar la URI completa. Referencia: conexión mediante drivers.

6. Guardar la configuración en el archivo privado

Copiar `.env.example` como `.env` y completar allí la URI real. En el apartado siguiente aparecen los archivos completos con **USUARIO, CLAVE y HOST como marcadores**, no como credenciales funcionales. Reemplazarlos antes de ejecutar la prueba de conexión. La plantilla pública conserva los marcadores.

La carpeta del proyecto continúa en la computadora. El `.env` también es un archivo local, aunque su URI apunte a una base en la nube. No subir ese archivo a GitHub ni mostrar la cadena privada en una captura.

7. Identificar las bases de los ejercicios

Podemos reutilizar el mismo clúster gratuito con bases distintas:

Actividad	Base	Colección
Lunes: Mercado artesanal	`mercado_artesanal_u7`	`productos`
Miércoles: caminatas	`caminatas_u7`	`inscripciones`
TP 07: salas	`reservas_salas_u7`	`reservas`

No hace falta crear las bases manualmente antes de escribir. La primera escritura confirmada permite empezar a inspeccionar los documentos. El plan Free tiene límites de capacidad y funciones; los ejemplos usan pocos registros. Referencia: límites del clúster gratuito.

La instalación local se explica en el anexo al final del apunte como alternativa de estudio. Para las clases y el TP seguimos con Atlas Free.

Estado final de los archivos de este paso

`package.json`

Este es el archivo completo de referencia. El ZIP usado en clase declara Mongoose `^9.11.0` y conserva otras versiones de Express, EJS y Morgan: no es necesario degradarlas ni reinstalarlas para resolver DNS. Para continuar ese proyecto, ejecutar `npm install` con su manifiesto y lockfile y conservar las versiones ya utilizadas. La versión 9.10.3 de este apunte sigue siendo la referencia para una copia nueva. Si la copia tiene otros metadatos, scripts o versiones ya instaladas, **conservarlos**; el cambio obligatorio es agregar `mongoose` mediante el comando anterior. No borrar dependencias que usa el sitio. Este manifiesto no agrega scripts de laboratorio: los ejecutaremos directamente con Node.

```
{
  "name": "mercado-artesanal-mongodb",
  "version": "1.0.0",
  "private": true,
  "type": "commonjs",
  "engines": {
    "node": ">=22.13.0"
  },
  "scripts": {
    "start": "node src/index.js",
    "start:local": "node --env-file=.env src/index.js",
    "format": "prettier --write src",
    "format:check": "prettier --check src",
    "lint": "eslint src",
    "check": "npm run format:check && npm run lint"
  },
  "dependencies": {
    "ejs": "^3.1.10",
    "express": "^5.1.0",
    "express-ejs-layouts": "^2.5.1",
    "mongoose": "9.10.3",
    "morgan": "^1.10.0"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "eslint": "^10.11.0",
    "prettier": "^3.9.9"
  }
}
```

`.env.example`

```
PORT=3000
NODE_ENV=development
MONGODB_URI=mongodb+srv://USUARIO:CLAVE@HOST/mercado_artesanal_u7?retryWrites=true&w=majority
```

`.env`

Crear este archivo local copiando `.env.example` . Su estructura completa queda así; sustituir los tres marcadores por los datos reales de Atlas **solo en este archivo privado**:

```
PORT=3000
NODE_ENV=development
MONGODB_URI=mongodb+srv://USUARIO:CLAVE@HOST/mercado_artesanal_u7?retryWrites=true&w=majority
```

`HOST` es el nombre completo que devuelve Atlas, incluido su dominio; `CLAVE` representa la contraseña con sus caracteres reservados codificados. Conservar además las opciones que tuviera la URI real. No imprimirla para demostrar la conexión. Si ya hay una variable del mismo nombre en el entorno de la terminal, Node la prioriza sobre el archivo; revisar esa situación si parece ignorar un cambio.

`.gitignore`

```
node_modules/
.env
```

Si había otras exclusiones, conservarlas. `.env.example` debe poder versionarse, `.env` no. Ignorar un archivo no retira uno que ya se publicó: si hubo una exposición, avisar al docente y reemplazar las credenciales.

Archivo generado: `package-lock.json` queda actualizado por NPM. Se entrega completo tal como lo genera la herramienta; no se incluye su listado extenso en el apunte.

Comprobación

```
npm ls mongoose
npm start
```

Debe aparecer Mongoose instalado y el sitio debe conservar su funcionamiento. Esto **todavía no prueba la conexión** a MongoDB. Detener el servidor web antes de continuar simplifica la observación de la terminal.

6. Paso 2 - Conectar y cerrar de manera controlada

Qué problema separa cada archivo

- `src/config/entorno.js` reúne la lectura del puerto, del modo de registro y de la URI. Leer configuración no abre conexiones.
- `src/config/base-datos.js` concentra apertura y cierre de conexión.
- `scripts/comprobar-conexion.js` coordina la prueba sin arrancar Express.

Primero crear `src/config/` y `scripts/` . Trasladar la función `leerConfiguracion` de `src/configuracion.js` a `src/config/entorno.js` y agregar allí `leerUriMongo` . Después cambiar en `src/index.js` la importación a `./config/entorno` . Buscar otras importaciones del módulo anterior y actualizar sus rutas relativas, si existen. Solo entonces eliminar `src/configuracion.js` : su función se conserva, no se duplica.

El servidor HTTP llama únicamente a `leerConfiguracion` ; por eso no exige una URI de MongoDB para arrancar. El laboratorio llama a `leerUriMongo` a través del módulo de conexión. Ninguna lectura se ejecuta al

importar el módulo.

Finalmente ampliar los comandos de Prettier y ESLint a `src scripts` , y el patrón de archivos de `eslint.config.js` a ambas carpetas. Mantener otros scripts y reglas propios de la copia. Los archivos completos de este paso incluyen también `src/index.js` , `package.json` y `eslint.config.js` .

`mongoose.connect()` es asíncrono. `await` espera su resultado antes de anunciar éxito. Si falla, el flujo salta fuera del `try` ; `finally` cierra los recursos y el `catch` final informa el fallo. `process.exitCode = 1` indica que la ejecución no terminó correctamente sin forzar una salida abrupta.

La selección del servidor tiene un plazo de 5000 ms para hacer visible un fallo de conexión; no es una garantía de duración total de cualquier problema de red. Mostramos el nombre del error, no toda la URI ni objetos que pudieran contener secretos. Referencia: conexiones de Mongoose.

Estado final de los archivos de este paso

src/config/entorno.js

```
function leerConfiguracion(entorno = process.env) {
  const puerto = Number(entorno.PORT ?? 3000);
  if (!Number.isInteger(puerto) || puerto < 1 || puerto > 65535) {
    throw new Error("PORT debe ser un entero entre 1 y 65535");
  }
  const formatoRegistro =
    entorno.NODE_ENV === "production" ? "combined" : "dev";
  return { puerto, formatoRegistro };
}
function leerUriMongo(entorno = process.env) {
  const uri = entorno.MONGODB_URI?.trim();
  if (!uri) {
    throw new Error("Falta MONGODB_URI en el entorno.");
  }
  if (!uri.startsWith("mongodb://") && !uri.startsWith("mongodb+srv://")) {
    throw new Error(
      "MONGODB_URI debe comenzar con mongodb:// o mongodb+srv://.",
    );
  }
  return uri;
}
module.exports = { leerConfiguracion, leerUriMongo };
```

src/config/base-datos.js

```
const mongoose = require("mongoose");
const dns = require("node:dns");
const { leerUriMongo } = require("./entorno");

// Corrección utilizada en el aula para las consultas DNS de Atlas.
dns.setServers(["8.8.8.8", "1.1.1.1"]);

async function conectarBaseDatos() {
  await mongoose.connect(leerUriMongo(), {
    serverSelectionTimeoutMS: 5000,
  });
}

async function desconectarBaseDatos() {
  await mongoose.disconnect();
}

module.exports = { conectarBaseDatos, desconectarBaseDatos };
```

scripts/comprobar-conexion.js

```
const {
  conectarBaseDatos,
  desconectarBaseDatos,
} = require("../src/config/base-datos");
async function main() {
  try {
    await conectarBaseDatos();
    console.log("Conexión con MongoDB establecida.");
  } finally {
    await desconectarBaseDatos();
  }
}
main().catch((error) => {
  console.error("No se pudo completar la conexión:", error.name);
  const codigosDns = [
    "ECONNREFUSED",
    "ENOTFOUND",
    "ETIMEOUT",
    "ESERVFAIL",
    "EAI_AGAIN",
  ];
  if (codigosDns.includes(error.code)) {
    console.error("Código DNS/red:", error.code);
  }
  if (error.syscall === "querySrv" || error.syscall === "queryTxt") {
    console.error("Consulta DNS:", error.syscall);
  }
  console.error("Revisar MONGODB_URI, servidor, red y permisos.");
  process.exitCode = 1;
});
```

src/index.js

```
const path = require("node:path");
const { leerJson } = require("./archivos");
const { leerConfiguracion } = require("./config/entorno");
const { crearServicioProductos } = require("./servicios/productos");
const { crearApp } = require("./app");
const rutaDatos = path.join(__dirname, "..", "datos", "productos.json");
async function main() {
  const { puerto, formatoRegistro } = leerConfiguracion();
  const productosIniciales = await leerJson(rutaDatos);
  const servicioProductos = crearServicioProductos(productosIniciales);
  const app = crearApp({ servicioProductos, formatoRegistro });
  app.listen(puerto, () => {
    console.log(`Aplicación disponible en http://localhost:${puerto}`);
  });
}
main().catch((error) => {
  console.error("No se pudo iniciar la aplicación:", error);
  process.exitCode = 1;
});
```

`package.json`

```
{
  "name": "mercado-artesanal-mongodb",
  "version": "1.0.0",
  "private": true,
  "type": "commonjs",
  "engines": {
    "node": ">=22.13.0"
  },
  "scripts": {
    "start": "node src/index.js",
    "start:local": "node --env-file=.env src/index.js",
    "format": "prettier --write src scripts",
    "format:check": "prettier --check src scripts",
    "lint": "eslint src scripts",
    "check": "npm run format:check && npm run lint"
  },
  "dependencies": {
    "ejs": "^3.1.10",
    "express": "^5.1.0",
    "express-ejs-layouts": "^2.5.1",
    "mongoose": "9.10.3",
    "morgan": "^1.10.0"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "eslint": "^10.11.0",
    "prettier": "^3.9.9"
  }
}
```

`eslint.config.js`

Si se continúa el ZIP de clase, su `check` solo revisa la sintaxis de dos archivos y no tiene instaladas las herramientas de calidad. Conservar las dependencias de ejecución y el lockfile, instalar `npm install --save-dev prettier eslint @eslint/js` y agregar los comandos de calidad mostrados en `package.json` ; NPM actualizará el lockfile. Tener `eslint.config.js` no instala ESLint por sí solo. Se puede conservar `start` con `--watch` ; para la prueba de MongoDB ejecutar el script directamente, sin watch.

```
const js = require("@eslint/js");

module.exports = [
  {
    files: ["src/**/*.js", "scripts/**/*.js"],
    ...js.configs.recommended,
    languageOptions: {
      sourceType: "commonjs",
      globals: {
        __dirname: "readonly",
        console: "readonly",
        module: "readonly",
        process: "readonly",
        require: "readonly",
      },
    },
  },
];
```

La corrección de DNS utilizada en clase

En el proyecto `ejemplo01-u07-c01` trabajado con el grupo se agregó `node:dns` y se ejecutó `dns.setServers(["8.8.8.8", "1.1.1.1"])` antes de conectar. El archivo completo de conexión mostrado arriba ya incorpora ese cambio: no hace falta crear otra carpeta, cambiar el modelo ni instalar un paquete DNS.

DNS traduce nombres y permite consultar registros. Con una URI `mongodb+srv://` , el driver consulta registros **SRV** para descubrir los servidores y **TXT** para obtener opciones. Una falla como `querySrv ECONNREFUSED`

indica que falló esa consulta DNS; no demuestra que Atlas haya rechazado la contraseña. Los mensajes `ENOTFOUND` , `ETIMEOUT` o `ESERVFAIL` también requieren revisar host, DNS y red según la operación que falló.

La lista configura servidores DNS públicos para las consultas de resolución de este proceso Node.js: 8.8.8.8 y 1.1.1.1. **No cambia el DNS de Windows, del navegador o de Compass**, ni abre permisos en Atlas. Afecta las funciones `dns.resolve*` , no `dns.lookup` ; por eso no garantiza resolver todos los problemas de nombres o conexión. Tampoco se debe asumir que el segundo servidor se probará ante cualquier respuesta negativa.

Conservamos la solución usada en el aula como base de estos ejemplos. Utilizarla solo si la red permite esos DNS y su uso está autorizado. Si la institución exige sus propios resolutores, acordar con el docente cuáles usar o retirar esa línea para usar la configuración predeterminada en una red donde funcione. No modificar la red del equipo ni eludir restricciones institucionales para completar el ejercicio.

Cómo corroborar la corrección:

1. Confirmar que el host de la URI es el real copiado de Atlas; no probar con el marcador HOST.
2. Ejecutar la prueba de conexión y distinguir nombre del error, código DNS y operación. La URI y las credenciales permanecen privadas.
3. Ante un fallo SRV/TXT, comprobar la línea DNS antes de `mongoose.connect()` y repetir en un **proceso nuevo**. Si conecta y termina, registrar ese resultado; no volver a ejecutar escrituras como prueba de conexión.
4. Si falla la autenticación, revisar usuario de base y contraseña; si hay un tiempo de espera de selección de servidor, revisar también IP permitida, disponibilidad del clúster y acceso de red. No atribuir todo error a DNS.
5. Documentar la solución y el entorno probado, sin afirmar que funciona en redes que no se ensayaron.

El ZIP mostraba `error.message` para investigar el fallo. En el material compartible conservamos el nombre y añadimos solo códigos DNS reconocidos y la operación SRV/TXT: un mensaje completo puede contener información del entorno y no debe publicarse sin revisión. El mensaje detallado puede consultarse en privado con el docente, retirando secretos antes de compartirlo.

Si los DNS públicos no están permitidos o la consulta sigue fallando, coordinar una red autorizada o consultar en Atlas la disponibilidad de una cadena estándar no SRV. Esa alternativa debe copiarse completa desde Atlas, con sus hosts y opciones; **no alcanza con cambiar** `mongodb+srv://` **por** `mongodb://` , ni conviene inventar IPs o desactivar TLS.

Referencias: Node.js: configuración de servidores DNS y Atlas: diagnóstico de conexión y DNS.

Comprobación

Desde la raíz de la copia:

```
node --env-file=.env scripts/comprobar-conexion.js
```

Resultado esperado: «Conexión con MongoDB establecida.» y regreso al prompt. No se crea ningún producto. Si la base todavía no aparece en Compass, eso no invalida la conexión: conectarse no equivale a guardar datos.

Prueba controlada de configuración: guardar la URI válida de manera privada y dejar temporalmente `MONGODB_URI=` vacía en `.env` . Repetir: debe fallar sin anunciar conexión. Restaurar la URI válida y probar otra vez. Si la terminal ya tiene esa variable definida, quitarla de ese entorno para que el ensayo use el archivo. Esta prueba demuestra la detección de configuración ausente; no simula una caída de Atlas.

Prueba controlada de autenticación: en ese mismo archivo privado, sustituir temporalmente solo la contraseña de la URI por un texto incorrecto sin caracteres reservados. Repetir la conexión: debe fallar sin anunciar éxito. Restaurar la URI válida y comprobar que vuelve a conectar. No modificar el usuario ni su contraseña en el panel de Atlas para este ensayo. Así distinguimos configuración ausente, autenticación rechazada y validación de los campos del producto.

Punto de control: no seguir con escrituras hasta que esta prueba funcione.

7. Paso 3 - Definir el esquema y construir el modelo

Por qué un archivo de modelo

Ya tenemos conexión, pero todavía no explicamos qué es un producto válido. Guardamos esa definición en `src/modelos/producto.js` para que no dependa del formulario ni de una ruta HTTP.

En este laboratorio todos los campos son obligatorios. `trim` elimina espacios en extremos; `Number` expresa el tipo esperado; `min: 1` establece un precio mínimo de 1 para esta práctica. No estamos diseñando todavía un sistema contable ni imponiendo dos decimales. Mongoose puede convertir ciertos valores: por ejemplo, un texto numérico puede convertirse a número; un texto no numérico falla. No confundir conversión con validación.

Elegimos explícitamente la colección `productos` para que el nombre no dependa de reglas de pluralización. `Producto` es el nombre del modelo en Node; `productos` es la colección en MongoDB.

La prueba de este paso construye un documento y llama a `validate()` . **No conecta y no guarda.** Así distinguimos reglas de datos de problemas de red. Las reglas presentadas aquí son locales; esta afirmación no se generaliza a validadores personalizados que consulten otros servicios.

Estado final de los archivos de este paso

`src/modelos/producto.js`

```
const mongoose = require("mongoose");

const productoSchema = new mongoose.Schema(
  {
    nombre: { type: String, required: true, trim: true },
    categoria: { type: String, required: true, trim: true },
    precio: { type: Number, required: true, min: 1 },
    descripcion: { type: String, required: true, trim: true },
  },
  { collection: "productos" },
);

const Producto = mongoose.model("Producto", productoSchema);

module.exports = { Producto };
```

`scripts/validar-producto.js`

```
const { Producto } = require("../src/modelos/producto");

async function main() {
  const producto = new Producto({
    nombre: "Mate de cerámica",
    categoria: "Cerámica",
    precio: 12000,
    descripcion: "Pieza artesanal esmaltada.",
  });
  await producto.validate();
  console.log("Datos válidos; todavía no guardamos en MongoDB.");
}

main().catch((error) => {
  console.error("Validación rechazada:", error.name);
  if (error.errors) {
    console.error("Campos a revisar:", Object.keys(error.errors).join(", "));
  }
  process.exitCode = 1;
});
```

Comprobación

```
node scripts/validar-producto.js
```

Debe imprimir «Datos válidos; todavía no guardamos en MongoDB.». Para comprobar el rechazo, cambiar temporalmente `precio: 12000` por `precio: -1` en ese script: debe informar `ValidationError` y el campo `precio` . Probar también nombre vacío, descripción ausente y precio `"abc"` , de a uno. Después de las pruebas, **restaurar el archivo completo mostrado arriba**.

Ninguna de estas ejecuciones inserta documentos. La validación del modelo complementará la validación del formulario cuando integremos HTTP; no reemplaza mensajes amigables, parsers ni controles del controlador. Referencia: validación en Mongoose.

Reglas que reutilizaremos el miércoles y en el TP

El contrato depende del dominio. Además de `required` , `trim` y `min` , presentamos estas opciones del esquema:

Opción	Ejemplo conceptual	Qué comprueba
`max`	`max: 5`	Límite superior de un número
`enum`	`enum: ["A", "B"]`	Pertenencia a una lista de textos permitidos
`match`	`match: /@/`	Presencia de @; no valida por completo un email
`validate`	`validate: { validator: Number.isInteger, message: "Debe ser entero" }`	Ejecuta una función que devuelve verdadero o falso

`Number.isInteger` ya se usaba en el middleware: ahora entregamos esa función al esquema, sin llamarla en la definición. Mongoose la ejecuta sobre el valor convertido. Un número 2.5 cumple un rango de 1 a 5, pero falla la regla de entero. Las fechas de caminatas y reservas seguirán siendo textos obligatorios como en los proyectos previos; todavía no validamos el calendario ni zonas horarias. No copiar estas opciones indiscriminadamente al precio: un precio puede tener decimales.

8. Paso 4 - Guardar el primer producto

Qué agregamos y por qué

Creamos una entrada dedicada a escribir. Importa el modelo, conecta y ejecuta `await Producto.create(...)` . `create` valida y guarda: el mensaje de éxito se imprime **después** de esa operación. No usamos `push` , no reescribimos el JSON y no calculamos un ID numérico.

El programa tiene datos ficticios fijos para observar una sola operación. **Cada ejecución válida crea un producto nuevo**, aunque repita el nombre. No hemos declarado unicidad ni diseñado un importador de semillas. No colocar esta creación en `base-datos.js` ni ejecutarla automáticamente al iniciar el sitio.

El cierre en `finally` es apropiado para este programa corto. En un servidor web que atiende solicitudes no debemos abrir y cerrar la conexión por cada pedido; esa integración se abordará en la unidad 8.

Estado final de los archivos de este paso

`scripts/crear-producto.js`

```
const {
  conectarBaseDatos,
  desconectarBaseDatos,
} = require("../src/config/base-datos");
const { Producto } = require("../src/modelos/producto");
async function main() {
  try {
    await conectarBaseDatos();
    const producto = await Producto.create({
      nombre: "Mate de cerámica",
      categoria: "Cerámica",
      precio: 12000,
      descripcion: "Pieza artesanal esmaltada.",
    });
    console.log("Producto guardado. _id:", producto._id.toString());
  } finally {
    await desconectarBaseDatos();
  }
}
main().catch((error) => {
  console.error("No se pudo crear el producto:", error.name);
  if (error.errors) {
    console.error("Campos a revisar:", Object.keys(error.errors).join(", "));
  }
  console.error("Revisar datos, configuración y disponibilidad de MongoDB.");
  process.exitCode = 1;
});
```

Comprobación

```
node --env-file=.env scripts/crear-producto.js
```

Anotar el `_id` impreso. Debe volver el prompt: el proceso terminó. En el proyecto de Atlas, abrir **Data Explorer** (también puede aparecer como **Browse Collections**), seleccionar `mercado_artesanal_u7` , colección `productos` , y actualizar la vista. Localizar el documento por su `_id` . No insertar otro desde el panel para reemplazar la escritura del programa. Compass sigue siendo opcional. Referencia: inspeccionar colecciones en Atlas.

Si no aparece el mensaje de guardado, no dar por hecha la escritura. Revisar primero la conexión y luego los campos. No borrar datos para «hacer que funcione».

9. Paso 5 - Leer desde una nueva ejecución y demostrar persistencia

Por qué separar lectura de creación

Si un programa crea un producto cada vez que lista, no permite demostrar que recupera un dato anterior. Este script solo consulta. `find({})` significa «buscar todos los documentos», y devuelve una consulta que esperamos con `await` . Una colección sin resultados produce un arreglo vacío; no es un error de conexión.

Convertimos `_id` a texto solo para mostrarlo en la terminal. No cambiamos el tipo almacenado. La tabla no incluye todos los campos: el documento sigue teniendo `descripcion` y puede contener `__v` .

Estado final de los archivos de este paso

`scripts/listar-productos.js`

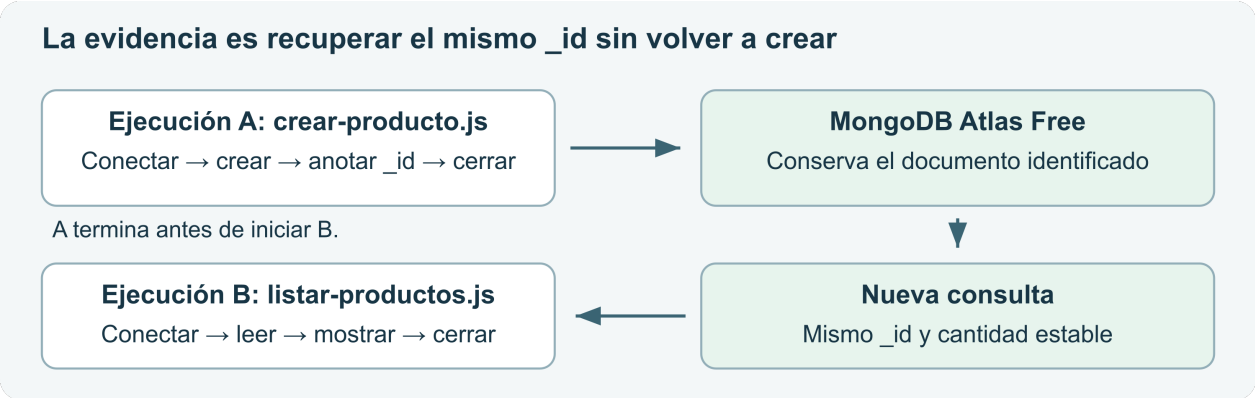
```
const {
  conectarBaseDatos,
  desconectarBaseDatos,
} = require("../src/config/base-datos");
const { Producto } = require("../src/modelos/producto");
async function main() {
  try {
    await conectarBaseDatos();
    const productos = await Producto.find({});
    console.table(
      productos.map((producto) => ({
        id: producto._id.toString(),
        nombre: producto.nombre,
        categoria: producto.categoria,
        precio: producto.precio,
      })),
    );
    console.log("Total de productos:", productos.length);
  } finally {
    await desconectarBaseDatos();
  }
}
main().catch((error) => {
  console.error("No se pudo consultar los productos:", error.name);
  console.error("Revisar configuración y disponibilidad de MongoDB.");
  process.exitCode = 1;
});
```

Comprobación reproducible

1. Ejecutar el listado y anotar la cantidad inicial. Si la base es nueva, puede ser 0; si ya se usó, no borrar registros.
2. Ejecutar la creación **una vez** y anotar su `_id` .
3. Esperar a que vuelva el prompt. Cerrar y abrir la terminal si se desea; mantener acceso a Internet y al clúster de Atlas.
4. Ejecutar el listado desde la raíz del mismo proyecto y con la misma URI.
5. Buscar el `_id` anotado. Debe estar y la cantidad debe haber aumentado en uno.
6. Ejecutar el listado otra vez: debe conservar la cantidad si nadie más escribe en esa base.

```
node --env-file=.env scripts/listar-productos.js
node --env-file=.env scripts/crear-producto.js
node --env-file=.env scripts/listar-productos.js
node --env-file=.env scripts/listar-productos.js
```

Si ya creaste un producto en el paso 4, esta secuencia creará **otro**. No hay una cantidad absoluta obligatoria: comparamos antes/después e identidad. El orden del listado no forma parte del contrato; no buscar el registro suponiendo que será la última fila.



Primera ejecución: conectar, guardar y cerrar. Segunda ejecución: conectar, consultar y recuperar el mismo identificador.

Prueba de escritura inválida: cambiar temporalmente el precio a -1 en `crear-producto.js` , ejecutarlo y volver a listar. Debe informar el error, terminar sin éxito y no agregar un documento. Restaurar el script al estado del paso 4. No se trata solo de que la terminal muestre un mensaje: la cantidad debe permanecer igual.

Qué demostramos: los datos sobreviven al cierre de Node.js. No hemos probado copias de seguridad, fallos de disco o recuperación ante desastres.

10. Cómo queda el proyecto y qué no cambió

```
mercado-artesanal-mongodb/
  datos/                        (sin cambios; semilla del sitio)
  public/                       (sin cambios)
  views/                        (sin cambios)
  src/
    index.js                    (importación actualizada; arranca Express)
    app.js                      (sin cambios)
    archivos.js                 (sin cambios)
    config/
      entorno.js                (PORT, NODE_ENV y MONGODB_URI)
      base-datos.js             (apertura y cierre)
    modelos/
      producto.js               (nuevo; esquema y modelo)
    rutas/                      (sin cambios)
    controladores/              (sin cambios)
    servicios/                  (sin cambios; datos en memoria)
    middleware/                 (sin cambios)
  scripts/
    comprobar-conexion.js       (no inserta)
    validar-producto.js         (no conecta ni inserta)
    crear-producto.js           (inserta una vez por ejecución)
    listar-productos.js         (solo lee)
  .env                          (local; no se entrega)
  .env.example                  (ampliado)
  .gitignore                    (conservar exclusiones)
  eslint.config.js              (incluye src y scripts)
  package.json                  (Mongoose y calidad en src/scripts)
  package-lock.json             (actualizado por NPM)
```

El antiguo `src/configuracion.js` se eliminó después de trasladar su función. Las herramientas de calidad ahora recorren tanto `src` como `scripts` , según la configuración completa del paso 2:

```
npm run format
npm run check
npm start
```

Comprobar que el sitio conserva inicio, listado, POST válido/inválido y 404. Los nuevos documentos del laboratorio no tienen por qué coincidir con el arreglo del sitio. **Todavía hay dos recorridos separados**, documentados a

propósito, no una migración incompleta que debamos ocultar.

11. Matriz de verificación y dificultades frecuentes

Caso	Resultado esperado	Qué revisar si falla
Conexión válida	Mensaje de éxito y proceso terminado	Servicio, URI, red y permisos
URI vacía o prefijo incorrecto	Error, sin mensaje de éxito	Archivo .env y variables heredadas
Modelo válido sin conexión	Validación correcta, sin escritura	Campos del objeto
Precio negativo en validación	ValidationError	Regla min del esquema
Creación válida	Nuevo _id y escritura confirmada	Esperar create con await
Nuevo proceso de listado	Recupera el mismo _id	Misma base y colección
Listar dos veces	Cantidad estable	No llamar create en el listado
Escritura inválida	Error y cantidad sin cambios	Consultar después, no solo leer el error
Sitio HTTP	Funciona como en unidad 6	No haber reemplazado su servicio

- `Cannot find module 'mongoose'` : comprobar directorio de trabajo y `npm install` en la copia.
- `querySrv ECONNREFUSED` , `queryTxt` u otro fallo DNS: seguir el diagnóstico del paso 2 y comprobar los resolutores autorizados antes de revisar el modelo.
- `MongooseServerSelectionError` : revisar disponibilidad del clúster de Atlas, IP permitida, conexión a Internet y acceso de la red. Si falla la autenticación, comprobar el usuario de base, contraseña y codificación. No publicar la URI para pedir ayuda.
- El programa no termina: revisar `await desconectarBaseDatos()` en `finally` .
- Aparece un objeto Query y no los datos: falta esperar `find` .
- Atlas y Node muestran cosas distintas: comparar clúster, base y colección, y actualizar Data Explorer. Para comparar cantidades, usar el listado de Node; el contador del panel puede estar basado en información almacenada en caché.
- Hay nombres repetidos: se ejecutó la creación varias veces; hoy el nombre no es único.
- La página no muestra el documento: el sitio todavía usa el arreglo de la unidad 6.
- `__v` apareció: es un campo de Mongoose, no un producto duplicado.

12. TP 07 y ticket de salida

El TP 07 aplica conexión, modelo, alta y consulta al dominio de reservas de salas, sobre una copia del TP 06. Se entrega individualmente por URL de GitHub el lunes siguiente a las 19:59. No pide todavía rutas REST persistentes, actualización, borrado ni autenticación.

Antes de terminar, responder:

1. ¿Qué diferencia hay entre instalar Mongoose y tener MongoDB disponible?
2. ¿Por qué un _id en un objeto no prueba que se haya guardado?
3. ¿Qué hace validate que no hace create, y viceversa?
4. ¿Por qué la consulta se ejecuta en otro proceso para demostrar persistencia?
5. ¿Qué archivos de la unidad 6 dejamos intactos y por qué?
6. ¿Qué habría que adaptar antes de conectar este modelo a los controladores HTTP?

Referencias oficiales

Las fuentes están enlazadas en cada explicación. Para preparar el entorno, consultar también los requisitos de Mongoose 9 y la instalación de MongoDB por sistema operativo.

Anexo - Cómo instalar MongoDB localmente

Esta explicación se conserva para conocer la alternativa local. **Las clases y el TP 07 utilizan Atlas Free.** Tener MongoDB local instalado no es un requisito de esta unidad.

1. Comprobar la compatibilidad de la máquina con la versión elegida. La guía oficial de MongoDB Community 8.0 para Windows contempla Windows 11 o Windows Server 2022 de 64 bits en x86_64.
2. Descargar el instalador oficial, elegir instalación completa y la opción de instalar MongoDB como servicio.
3. Comprobar en Servicios de Windows que MongoDB está en ejecución. Mantenerlo accesible solo desde la máquina local.
4. Compass es opcional; `mongosh` se instala por separado. En macOS o Linux seguir la guía correspondiente al sistema.
5. Para un ensayo local separado, la URI sería `mongodb://127.0.0.1:27017/mercado_artesanal_u7` : protocolo, equipo local, puerto y base. `127.0.0.1` evita una posible resolución de `localhost` a IPv6.

Los modelos y las operaciones de Mongoose siguen siendo los mismos; cambia el servidor indicado por la URI. Los documentos guardados localmente no se trasladan automáticamente a Atlas. Al retomar las clases, restaurar en `.env` la URI privada de Atlas.